OtterSec

Atomic Liquidity Vault

Security Assessment

Prepared for
Ember Protocol

Prepared by
Otter Audits, LLC

Prepared on
October 5th, 2026

Michał Bochnak
embe221ed@osec.io

Sangsoo Kang
sangsoo@osec.io

Table of Contents

1. Executive Summary 3
1.1. Overview 3
1.2. Key Findings 3
2. Scope 4
3. Findings 5
3.1. Findings Summary 5
4. Advisories 6
OS-ALV-ADV-00 Missing SUI-Specific Suilend Redemption Path 7
OS-ALV-ADV-01 Incomplete Source Withdrawal Checks and Returned Receipt Accounting 9
OS-ALV-ADV-02 Framework Deny List Can Stall Withdrawal Queue 12
OS-ALV-ADV-03 Fixed-Point Rounding Undershoots Suilend Redemptions 14
OS-ALV-ADV-04 Unaccounted Source Withdrawal Fees in Atomic Swaps 16
OS-ALV-ADV-05 Transferable Shares Bypass Address-Based Withdrawal Policies 19
OS-ALV-ADV-06 Transient TVL Dilution of Source Utilization Limits 21
OS-ALV-ADV-07 Partition-Dependent Tier Fees and Exposure Rounding 23
OS-ALV-ADV-08 Transient Source TVL Inflates Epoch Swap Capacity 24
OS-ALV-ADV-09 Nominal Payout Accounting Ignores Dust Shortfalls 26
OS-ALV-ADV-10 Missing Version Checks in Shared-Config Mutators 27
OS-ALV-ADV-11 Full Suilend Redemption Can Block Strategy Exit 28
OS-ALV-ADV-12 Annual Fee Undercollection from a Shared Fee Ledger 30
OS-ALV-ADV-13 Overflow in Platform Fee Accrual 33
5. Suggestions 34
OS-ALV-SUG-00 Stale Share Rates Misallocate Vault Backing 35
OS-ALV-SUG-01 Stale Source Rates Shift Losses to Atomic LPs 37
OS-ALV-SUG-02 Account-Wide Timestamp Distorts Time-Based Withdrawal Fees 38
OS-ALV-SUG-03 Incomplete Validation of Initial Vault Configuration 39
OS-ALV-SUG-04 Linear Account Lists Increase Operational Gas Costs 41
OS-ALV-SUG-05 Misleading Net Amount for Holdback-Waived Swaps 43
OS-ALV-SUG-06 Stale Sequence on Swap Auto-Pull Events 44
OS-ALV-SUG-07 Floor-Rounded TVL Weakens the Maximum Cap 45
OS-ALV-SUG-08 Overflow-Prone Solvency Margin View 47

© 2026 Otter Audits LLC. All Rights Reserved.

OS-ALV-SUG-09 Redundant Source Identifier in Held-Value Helper 48
OS-ALV-SUG-10 Suilend Market Identity Not Committed at Activation 49
OS-ALV-SUG-11 Unauthenticated Sub-Account Attribution for Vault Deposits 52
OS-ALV-SUG-12 Missing Witness for Atomic Vault Sharing 54
OS-ALV-SUG-13 Backing Queries May Abort or Mislead 55
A. Vulnerability Rating Scale 56
B. Procedure 57

© 2026 Otter Audits LLC. All Rights Reserved.

1. Executive Summary

1.1. Overview

Ember Protocol engaged OtterSec to conduct a security assessment of the `Atomic Liquidity Vault` program. This assessment was conducted between August 24th and August 30th, 2026. Follow-up reviwes, inlcuding fix reviews were performed between September 4th and September 29th, 2026. For more information on our auditing methodology, refer to Appendix B.

1.2. Key Findings

We produced 28 findings throughout this audit engagement.

Among the most impactful vulnerabilities we identified, one concerns the absence of a SUI-specific Suilend redemption path: a SUI atomic vault can supply Suilend, but every withdrawal, swap, and strategy-clear path lacks the SuiSystemState-based unstaking flow, so these operations abort whenever the payout depends on staked SUI (● OS-ALV-ADV-00).

Additionally, the EmberShare swap guard checks only the source vault's legacy pause flag rather than its combined granular withdrawal-pause state, and returned receipt coins from skipped redemptions vanish from held-exposure accounting until reclaimed. As a result, swaps remain open while the atomic vault cannot redeem at the source, and utilization headroom is temporarily reopened for further swaps (● OS-ALV-ADV-01). Furthermore, neither vault consults Sui's framework deny list before enqueuing or settling withdrawals, so a request with an externally prohibited receiver aborts settlement and pins the FIFO head, stalling every later withdrawal until privileged intervention (● OS-ALV-ADV-02).

We also found that the Suilend withdrawal helper applies its ceiling after a precision-losing fixed-point division, which can select one cToken too few and trip the short-delivery assertion (● OS-ALV-ADV-03). Also, because vault shares are freely transferable while time-based fees, exemptions, and blacklist status are keyed to addresses, a holder can route shares through a cooperating address to escape the fee window or a blacklisting (● OS-ALV-ADV-05). Similarly, the source-utilization cap divides by the atomic vault's instantaneous TVL, so a depositor can inflate the denominator just before a swap and withdraw immediately after, leaving remaining LPs with source concentration above the configured limit (● OS-ALV-ADV-06).

We further highlighted that the dust-tolerant payment helper's actual transfer is discarded in favor of nominal amounts across all swap path, allowing a receiver to be paid up to 1,000 base units less than the slippage check validated and the events report (● OS-ALV-ADV-09). Finally, the platform-fee accrual multiplies TVL, fee percentage, and elapsed time into a single u128 numerator that can overflow, and since the checkpoint timestamp only advances on success, the abort recurs and can repeatedly block deposits, withdrawals, and fee operations (● OS-ALV-ADV-13).

© 2026 Otter Audits LLC. All Rights Reserved.

2. Scope

The `Atomic Liquidity Vault` is a new vault type offering instant withdrawals and immediate swaps of classic-vault receipt shares for collateral, with idle funds deployable into Suilend yield strategies.

The source code was delivered to us in a Git repository at `https://github.com/fireflyprotocol/ember-vaults`.

The audit was performed against PR#37 and PR#67.

© 2026 Otter Audits LLC. All Rights Reserved.

3. Findings

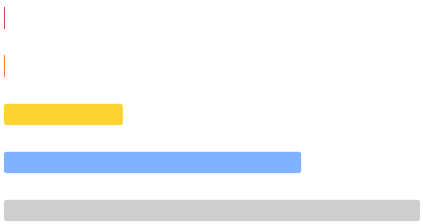
Findings were rated using the criteria in Appendix A.

We split the findings into **advisories** and **suggestions**. Advisories have an immediate impact and should be remediated as soon as possible. Suggestions do not have an immediate impact but will aid in mitigating future vulnerabilities.

Items are tracked with a stable identifier: `OS-ALV-ADV-*` for advisories and `OS-ALV-SUG-*` for suggestions.

3.1. Findings Summary

Severity	Count
Critical	0
High	0
Medium	4
Low	10
Informational	14



© 2026 Otter Audits LLC. All Rights Reserved.

4. Advisories

Here, we present a technical analysis of the 14 vulnerabilities we identified during our audit. These vulnerabilities have **immediate** security implications, and we recommend remediation as soon as possible.

ID	Severity	Status	Description
OS-ALV-ADV-00	Medium	✓ Resolved	Missing SUI-Specific Suilend Redemption Path
OS-ALV-ADV-01	Medium	✓ Resolved	Incomplete Source Withdrawal Checks and Returned Receipt Accounting
OS-ALV-ADV-02	Medium	○ Ack'd	Framework Deny List Can Stall Withdrawal Queue
OS-ALV-ADV-03	Medium	✓ Resolved	Fixed-Point Rounding Undershoots Suilend Redemptions
OS-ALV-ADV-04	Low	○ Ack'd	Unaccounted Source Withdrawal Fees in Atomic Swaps
OS-ALV-ADV-05	Low	○ Ack'd	Transferable Shares Bypass Address-Based Withdrawal Policies
OS-ALV-ADV-06	Low	✓ Resolved	Transient TVL Dilution of Source Utilization Limits
OS-ALV-ADV-07	Low	○ Ack'd	Partition-Dependent Tier Fees and Exposure Rounding
OS-ALV-ADV-08	Low	○ Ack'd	Transient Source TVL Inflates Epoch Swap Capacity
OS-ALV-ADV-09	Low	✓ Resolved	Nominal Payout Accounting Ignores Dust Shortfalls
OS-ALV-ADV-10	Low	✓ Resolved	Missing Version Checks in Shared-Config Mutators
OS-ALV-ADV-11	Low	○ Ack'd	Full Suilend Redemption Can Block Strategy Exit
OS-ALV-ADV-12	Low	○ Ack'd	Annual Fee Undercollection from a Shared Fee Ledger
OS-ALV-ADV-13	Low	✓ Resolved	Overflow in Platform Fee Accrual

© 2026 Otter Audits LLC. All Rights Reserved.

Missing SUI-Specific Suilend Redemption Path

ID	Severity	Status
OS-ALV-ADV-00	Medium	Resolved

Description

An `AtomicLiquidityVault<SUI>` can activate Suilend and supply its idle SUI because the Suilend setup and supply paths accept a generic asset type without rejecting SUI. When the reserve has moved physical SUI into its staker, redemption must request liquidity, unstake SUI through `SuiSystemState`, and then fulfill the request. However, `withdraw_from_suilend` calls the generic combined redemption helper and accepts neither `SuiSystemState` nor an equivalent SUI-specific execution path.

sources/strategy.move

```
public(package) fun withdraw_from_suilend<P, T>(
    uid: &mut UID,
    market: &mut LendingMarket<P>,
    clock: &Clock,
    target_amount: u64,
    ctx: &mut TxContext,
): Balance<T> {
    [...]
    let split_bal = balance::split(bal, to_redeem);
    let received = lending_market::redeem_ctokens_and_withdraw_liquidity<P, T>(
        market,
        reserve_index,
        clock,
        coin::from_balance(split_bal, ctx),
        option::none(),
        ctx,
    );
    [...]
}
```

The same incompatibility affects strategy teardown. As shown in `clear_suilend_state` below, clearing a non-empty Suilend position redeems all held cTokens through the same generic combined helper before removing the strategy state. Because this function also lacks a SUI-specific branch and a mutable `SuiSystemState` argument, it cannot perform the required unstaking step when the reserve's liquid SUI is insufficient.

sources/strategy.move

```
public(package) fun clear_suilend_state<P, T>(
    uid: &mut UID,
    market: &mut LendingMarket<P>,
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
    clock: &Clock,
    ctx: &mut TxContext,
): Balance<T> {
    let mut drained = balance::zero<T>();
    if (dynamic_field::exists_with_type<vector<u8>, Balance<CToken<P, T>>>(uid,
    KEY_SUILEND_CTOKENS)) {
        let bal: Balance<CToken<P, T>> = dynamic_field::remove(uid,
        KEY_SUILEND_CTOKENS);
        let held = balance::value(&bal);
        if (held > 0) {
            let reserve_index = *dynamic_field::borrow<vector<u8>, u64>(
                uid, KEY_SUILEND_RESERVE_INDEX,
            );
            let received = lending_market::redeem_ctokens_and_withdraw_liquidity<P, T>(
                market,
                reserve_index,
                clock,
                coin::from_balance(bal, ctx),
                option::none(),
                ctx,
            );
            balance::join(&mut drained, coin::into_balance(received));
        } else {
            balance::destroy_zero(bal);
        };
    };
    [...]
}
```

`withdraw_from_suilend` is used by explicit operator withdrawals and by the Suilend liquidity helper that serves instant withdrawals and EmberShare swaps. Consequently, operator pulls, instant withdrawals, Suilend-backed swaps, and strategy clearing abort whenever their payout depends on SUI held in the reserve's staker rather than its liquid balance.

Remediation

Route every SUI-denominated Suilend redemption through a dedicated request, unstake, and fulfill flow, while preventing generic redemption paths from handling SUI.

Patch

Resolved in e533539 and 98a0ab8.

© 2026 Otter Audits LLC. All Rights Reserved.

Incomplete Source Withdrawal Checks and Returned Receipt Accounting

ID	Severity	Status
OS-ALV-ADV-01	Medium	Resolved

Description

`do_swap_ember_shares_core` verifies the Atomic vault's withdrawal pause and blacklist state, but checks the source vault only through `vault::get_vault_paused`, which only returns the source vault's legacy `paused` field. It does not establish that withdrawals are enabled under the source's granular controls or that the Atomic object can redeem there before accepting receipt tokens and transferring immediate collateral. Consequently, the guard does not observe operation-specific withdrawal pauses stored as dynamic fields.

sources/atomic_liquidity_vault.move

```
fun do_swap_ember_shares_core<T, R>(
    atomic: &mut AtomicLiquidityVault<T>,
    config: &ProtocolConfig,
    source_vault: &Vault<T, R>,
    share_coin: Coin<R>,
    receiver: address,
    min_net_out: u64,
    clock: &Clock,
    ctx: &mut TxContext,
): (u64, u64, u64, u64, u64) {
    [...]
    // Refuse the swap when the source vault has its own operations paused.
    // Without this guard users could park source shares here even though
    // `redeem_shares` on the source is currently disabled — the atomic
    // vault would accumulate shares it cannot subsequently redeem at the
    // source until the source unpauses. `vault.move`'s `Vault` uses a
    // single unified `paused` flag (unlike the atomic vault's three
    // per-operation flags), so checking it covers the source's
    // withdrawal flow directly.
    assert!(!vault::get_vault_paused(source_vault), ESourceVaultWithdrawalsPaused);
    [...]
}
```

Actual source redemptions call `verify_withdrawals_not_paused`, whose underlying `is_withdrawals_paused` result combines the legacy pause with the granular withdrawal pause. A source configured with only the granular withdrawal pause therefore rejects redemption while the Atomic swap remains available.

© 2026 Otter Audits LLC. All Rights Reserved.

sources/vault.move

```
2047  public fun is_withdrawals_paused<T,R>(vault: &Vault<T,R>): bool {
2048      vault.paused || df_get_bool(&vault.id, KEY_PAUSED_WITHDRAWALS)
2049  }
```

Effective source withdrawal-pause state

`current_held_share_value_in_collateral`, which supplies the held value used by swap previews and the swap's absolute-cap and utilization checks, sums only receipt shares held in the Atomic vault's per-source dynamic field and shares recorded as pending under the Atomic object's address in the source vault.

sources/atomic_liquidity_vault.move

```
1   fun current_held_share_value_in_collateral<T, R>(
2       atomic: &AtomicLiquidityVault<T>,
3       source_vault: &Vault<T, R>,
4       source_id: ID,
5   ): u64 {
6       let key = CollectedEmberSharesKey { source_vault_id: source_id };
7       let custodied_shares = if (dynamic_field::exists_with_type<CollectedEmberSharesKey,
        Balance<R>>(&atomic.id, key)) {
8           balance::value(
9               dynamic_field::borrow<CollectedEmberSharesKey, Balance<R>>(&atomic.id,
                key),
10          )
11      } else { 0 };
12      [...]
13      let pending_shares = if (!vec_set::is_empty(&atomic.redemption_active_sources)
14          && vec_set::contains(&atomic.redemption_active_sources, &source_id)) {
15          let atomic_addr = object::uid_to_address(&atomic.id);
16          vault::get_account_total_pending_withdrawal_shares<T, R>(
17              source_vault, atomic_addr,
18          )
19      } else { 0 };
20      let total_shares = custodied_shares + pending_shares;
21      [...]
22  }
```

`redeem_collected_ember_shares` assigns the Atomic object's address as both owner and receiver of the source withdrawal request. Accordingly, it is the Atomic object's source-side blacklist status, rather than the original swapper's or payout receiver's status, that can cause the later redemption to be skipped. On the skip path, source processing returns the receipt coin to the request owner and calls `update_account_state` to remove the shares from pending accounting. The returned transfer-to-object coin is not present in either location counted by `current_held_share_value_in_collateral`.

© 2026 Otter Audits LLC. All Rights Reserved.

It becomes counted again only after the trusted operator invokes `claim_returned_ember_shares`, which receives the coin through the Atomic object and joins it back into the per-source custodied balance.

Receipt holders can therefore obtain immediate collateral while the source's granular withdrawal pause prevents the Atomic vault from initiating redemption. If a later redemption is skipped, Atomic liquidity providers remain exposed to the returned, temporarily illiquid receipt tokens while reported exposure temporarily falls, reopening utilization and absolute held-value headroom for another swap. The first swap's rolling epoch-volume usage remains recorded, so this accounting gap does not bypass the epoch-cap counter.

Proof of Concept

1. Leave the source vault's legacy `paused` field false and enable only its granular withdrawal pause. A direct source redemption is rejected by the combined withdrawal-pause check, while an otherwise valid Atomic swap observes the false legacy field and can transfer immediate collateral.
2. In separate state, use a first swap to fill the source's configured held-value limit, then submit the collected receipts for source redemption. Blacklist the Atomic object's address in the source before processing that request.
3. Source processing skips the request, removes its pending-share state, and returns the receipt coin to the Atomic object.
4. Before the trusted operator calls `claim_returned_ember_shares`, query or exercise the held-exposure checks. The returned coin is neither custodied in the counted dynamic field nor pending in the source, so the first position is omitted and a second otherwise eligible swap may pass the utilization and absolute held-value checks. The rolling epoch cap still includes the first swap.

Remediation

Replace the source check in `do_swap_ember_shares_core` with `vault::is_withdrawals_paused(source_vault)` and apply the same source-withdrawal check to every executable swap preview. Before accepting source receipts, derive the Atomic object's address and verify that it is eligible to act as both owner and receiver of a source redemption, including checking the source blacklist. Maintain a per-source redemption ledger from submission through final collateral or receipt claim.

Patch

Resolved in d7b4ad1, 75547f6, and 98a0ab8.

© 2026 Otter Audits LLC. All Rights Reserved.

Framework Deny List Can Stall Withdrawal Queue

ID	Severity	Status
OS-ALV-ADV-02	Medium	Ack'd

Description

For collateral type `T` governed by Sui's framework deny list, a user can queue a withdrawal whose receiver is prohibited from receiving `T`. The classic vault's `redeem_shares_for_owner` validates the zero address and Ember's internal blacklist, but does not consult the framework deny list before parking the shares and enqueuing the request.

sources/vault.move

```
public(package) fun redeem_shares_for_owner<T,R>(
    vault: &mut Vault<T,R>,
    config: &ProtocolConfig,
    shares: Balance<R>,
    owner: address,
    receiver: address,
    clock: &Clock,
): WithdrawalRequest {
    [...]
    // Reject null receiver up-front. `transfer::public_transfer(coin, @0x0)`
    // does NOT abort on Sui — it silently transfers to an orphaned account,
    // effectively burning the payout with no queue-head freeze to flag it.
    // Catching the invalid receiver at enqueue time surfaces the mistake
    // where it happens rather than losing funds at settlement.
    assert!(receiver != @0x0, EInvalidAccount);
    verify_not_blacklisted(vault, owner);
    verify_not_blacklisted(vault, receiver);
    [...]
}
```

`process_request` skips requests only when the owner or receiver is internally blacklisted, the request is cancelled, or the calculated payout is zero. Otherwise, it burns the pending shares and transfers a newly created `Coin<T>` to the receiver. Because the surrounding operator loop dequeues the request before calling `process_request`, a validator-level rejection of that transfer aborts the transaction and rolls back both the share burn and dequeue, leaving the same request at the FIFO head. The atomic vault has the same settlement gap. Its request-creation path likewise does not check the framework deny list, so an externally prohibited receiver can enter and remain at the head of its queue.

Consequently, routine processing attempts may consume gas without advancing either queue, delaying every later withdrawal. Classic-vault users have no alternative withdrawal path, while atomic-vault users can avoid the queue only when instant withdrawal is available and has sufficient capacity. An operator can recover by adding the request owner or receiver to Ember's

© 2026 Otter Audits LLC. All Rights Reserved.

internal blacklist so that settlement takes the skip path, but multiple pre-seeded requests could require repeated privileged intervention.

Remediation

Thread Sui's `DenyList` through the enqueue and settlement entry points for both vault types. At settlement, before burning or reducing pending shares, check the receiver with `coin::deny_list_v2_contains_current_epoch<T>` and the collateral type with `coin::deny_list_v2_is_global_pause_enabled_current_epoch<T>`. If the request cannot be paid, dequeue it, restore the shares to the owner, and emit an explicit skip reason. Retain settlement-time validation even if enqueue-time checks are added, because deny-list state may change while a request is queued. Also add an operator-only `force_skip_head` recovery path for external transfer failures that these checks cannot detect.

Patch

Acknowledged by the team.

© 2026 Otter Audits LLC. All Rights Reserved.

Fixed-Point Rounding Undershoots Suilend Redemptions

ID	Severity	Status
OS-ALV-ADV-03	Medium	Resolved

Description

`strategy::withdraw_from_suilend` intends to redeem enough cTokens to return at least `target_amount`. It computes the required cToken amount by applying `decimal::ceil` to the result of `decimal::div`, caps that amount by the vault's cToken balance, redeems it through Suilend, and requires the received balance to cover the target.

sources/strategy.move

```
public(package) fun withdraw_from_suilend<P, T>(
    uid: &mut UID,
    market: &mut LendingMarket<P>,
    clock: &Clock,
    target_amount: u64,
    ctx: &mut TxContext,
): Balance<T> {
    assert!(target_amount > 0, EZeroAmount);
    assert!(
        dynamic_field::exists_with_type<vector<u8>, Balance<CToken<P, T>>>(uid,
        KEY_SUILEND_CTOKENS),
        ESuilendStateMissing,
    );
    let reserve_index = *dynamic_field::borrow<vector<u8>, u64>(uid,
    KEY_SUILEND_RESERVE_INDEX);
    // Compute cTokens-needed via the simulated ratio (forward-projecting
    // accrued interest to `clock`). Round UP so we redeem ENOUGH to cover
    // target_amount; the on-chain redeem may return slightly more.
    let ctoken_amount = {
        let r = lending_market::reserve<P, T>(market);
        let ratio = reserve::simulated_ctoken_ratio<P>(r, clock);
        decimal::ceil(decimal::div(decimal::from(target_amount), ratio))
    };
    [...]
}
```

However, the ceiling is applied too late. Suilend's fixed-point division will have already rounded the quotient down to 18 decimals. When the exact quotient is slightly above an integer by less than one WAD quantum, the rounded quotient equals that integer and `decimal::ceil` leaves it unchanged. Suilend then floors the corresponding cToken-to-underlying conversion during redemption, so the selected amount can return one underlying base unit less than requested. The final `EStrategyShortDelivery` assertion consequently aborts the transaction.

© 2026 Otter Audits LLC. All Rights Reserved.

The affected helper is used by explicit operator withdrawals and by the Suilend liquidity helper that covers deficits for instant withdrawals and source-share swaps. These operations may therefore fail at the affected rounding boundaries even when the vault has sufficient cToken backing and the Suilend reserve has sufficient liquidity.

Remediation

Calculate and validate the cToken redemption amount according to Suilend's forward rounding behavior, while distinguishing insufficient backing from unexpected short delivery.

Patch

Resolved in fab37ee.

© 2026 Otter Audits LLC. All Rights Reserved.

Unaccounted Source Withdrawal Fees in Atomic Swaps

ID	Severity	Status
OS-ALV-ADV-04	Low	Ack'd

Description

`do_swap_ember_shares_core` values incoming receipt shares using the source vault's current rate, but deducts only the Atomic vault's swap fee and holdback. It neither incorporates the source vault's withdrawal fees into the quote nor verifies that the Atomic vault will be exempt from those fees.

sources/atomic_liquidity_vault.move

```
fun do_swap_ember_shares_core<T, R>(
    atomic: &mut AtomicLiquidityVault<T>,
    config: &ProtocolConfig,
    source_vault: &Vault<T, R>,
    share_coin: Coin<R>,
    receiver: address,
    min_net_out: u64,
    clock: &Clock,
    ctx: &mut TxContext,
): (u64, u64, u64, u64, u64) {
    [...]
    let now = clock::timestamp_ms(clock);
    let source_tvl = vault::get_vault_tvl(source_vault);
    // Floor rounding — see `preview_swap_ember_shares` for the rationale.
    let gross_amount = vault::calculate_amount_from_shares_floor(source_vault,
    share_amount);
    assert!(gross_amount > 0, EZeroAmount);
    [...]
    // Capped so fee + holdback can't exceed gross — protects against
    // misconfiguration where (base_fee * multiplier) + holdback > 100%.
    let fee_plus_holdback = fee_amount + holdback_amount_raw;
    assert!(fee_plus_holdback <= gross_amount, EInvalidPercentage);
    let net_amount = gross_amount - fee_plus_holdback;
    [...]
}
```

The source shares are not redeemed as part of the swap. Instead, `redeem_collected_ember_shares` later submits them to the source vault's normal withdrawal queue with the Atomic vault's address as both owner and receiver.

© 2026 Otter Audits LLC. All Rights Reserved.

sources/atomic_liquidity_vault.move

```
public fun redeem_collected_ember_shares<T, R>(
    atomic: &mut AtomicLiquidityVault<T>,
    config: &ProtocolConfig,
    source_vault: &mut Vault<T, R>,
    amount: u64,
    clock: &Clock,
    ctx: &mut TxContext,
) {
    [...]
    let atomic_addr = object::uid_to_address(&atomic.id);
    vault::redeem_shares_for_owner<T, R>(
        source_vault,
        config,
        split_bal,
        atomic_addr,
        atomic_addr,
        clock,
    );
    [...]
}
```

When the source operator processes the queued request, `process_request` evaluates the fee policy and exemption list that are effective at settlement time. A non-exempt request is charged the permanent fee and, when configured, the time-based fee. Because receipt shares transferred to the Atomic vault do not record a source deposit for the Atomic vault's address, its missing `last_deposit_ts` resolves to zero, which causes the time-based fee to apply.

The exemption is not a stable invariant. `set_fee_exemption_list` allows the source operator to add or remove the Atomic vault's address. The source module also exposes no public or package-level view through which the Atomic swap path could verify the effective withdrawal fees or exemption status. Consequently, an exemption may be absent when a swap is admitted or revoked before the source request is settled. The Atomic vault therefore values its obligations before source withdrawal fees while recovering collateral after those fees. Any unpriced difference reduces the collateral backing available for user payouts and accrued platform fees, and sufficiently large or repeated differences may leave accrued platform fees uncollectible.

Remediation

Expose source-vault views that return the effective permanent and time-based withdrawal fees, whether a supplied address is exempt, and a policy version suitable for detecting changes. Require the Atomic vault's address to be exempt when a source vault is admitted and before each swap, and recheck the condition when collected shares are submitted for redemption. If that exemption cannot be made stable through settlement, reserve a mandatory holdback sufficient to cover the worst-case source withdrawal fees and permitted rate movement.

© 2026 Otter Audits LLC. All Rights Reserved.

Patch

Acknowledged by the team.

© 2026 Otter Audits LLC. All Rights Reserved.

Transferable Shares Bypass Address-Based Withdrawal Policies

ID	Severity	Status
OS-ALV-ADV-05	Low	Ack'd

Description

Classic vault shares are transferable `Coin<R>` objects, but the policy state governing those shares is stored by address rather than attached to the economic claim. In `deposit_asset_v2`, the vault records the deposit timestamp under `ctx.sender()`, regardless of who ultimately receives or later holds the minted receipt coin.

sources/vault.move

```
1  public fun deposit_asset_v2<T,R>(vault: &mut Vault<T,R>, config: &ProtocolConfig,
   balance: Balance<T>, min_shares: u64, clock: &Clock, ctx: &mut TxContext): Coin<R> {
2      [...]
3      let owner = ctx.sender();
4      verify_not_blacklisted(vault, owner);
5      verify_not_sub_account(vault, owner);
6      // if deposit user list exists, only allow listed addresses to deposit
7      if(dynamic_field::exists_with_type<vector<u8>, vector<address>>(&vault.id,
       DEPOSIT_USER_LIST_DYNAMIC_FIELD)){
8          let deposit_user_list = dynamic_field::borrow<vector<u8>,
           vector<address>>(&vault.id, DEPOSIT_USER_LIST_DYNAMIC_FIELD);
9          assert!(vector::contains(deposit_user_list, &owner), EDepositNotAllowed);
10     };
11     [...]
12 }
```

The gateway makes this mismatch directly accessible by allowing the depositor to specify a different `receiver`. Its `deposit_asset_v2` wrapper obtains the receipt coin from the vault and publicly transfers it to the selected address, while the vault has recorded only the depositor's timestamp.

sources/gateway.move

```
436 entry fun deposit_asset_v2<T, R>(
437     vault: &mut Vault<T, R>,
438     config: &ProtocolConfig,
439     coin: Coin<T>,
440     min_shares: u64,
441     receiver: Option<address>,
442     clock: &Clock,
443     ctx: &mut TxContext
444 ){
445     let balance = coin::into_balance(coin);
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
446     let receipt_token = vault::deposit_asset_v2(vault, config, balance, min_shares,
        clock, ctx);
447     let send_to = if(option::is_some(&receiver)){
448         *option::borrow(&receiver)
449     } else {
450         ctx.sender()
451     };
452     transfer::public_transfer(receipt_token, send_to);
453 }
```

The minted receipt coin may be sent to another address

When shares are redeemed, `redeem_shares` identifies the owner as the current transaction sender. `redeem_shares_for_owner` checks that owner and the payout receiver against the blacklist and stores the owner in the withdrawal request. It does not retain the depositor or any provenance associated with the consumed receipt coins. At settlement, `process_request` applies blacklist checks, fee exemption, and the time-based withdrawal fee using `request.owner`. Consequently, the applicable policy is determined by the address that submitted the redemption rather than by the origin or transfer history of the shares.

A holder can therefore transfer recently minted shares to an address whose recorded deposit is outside the fee window, allowing the shares to avoid the time-based fee. Shares can similarly inherit a cooperating address's fee exemption, while a blacklisted holder can transfer them to a non-blacklisted address that can submit and settle a redemption.

Remediation

Bind provenance-sensitive withdrawal state to the shares themselves.

Patch

Acknowledged by the team.

© 2026 Otter Audits LLC. All Rights Reserved.

Transient TVL Dilution of Source Utilization Limits

ID	Severity	Status
OS-ALV-ADV-06	Low	Resolved

Description

`do_swap_ember_shares_core` calculates post-swap source utilization by dividing `new_held_value` by the atomic vault's current TVL. However, it assigns zero utilization whenever `vault_tvl` is zero, even when the vault will hold positive source exposure after the swap. The subsequent percentage-cap check therefore accepts that exposure when `max_utilization_percentage` is zero, and the fee multiplier is also selected using the zero-utilization value. The independent absolute held-value cap prevents this condition when configured appropriately, but a value of zero explicitly disables that cap.

sources/atomic_liquidity_vault.move

```
fun do_swap_ember_shares_core<T, R>([...]): (u64, u64, u64, u64, u64) {
    [...]
    // vault WILL hold once the swap completes.
    let current_held_value = current_held_share_value_in_collateral<T, R>(
        atomic, source_vault, source_id,
    );
    let new_held_value = current_held_value + gross_amount;
    let vault_tvl = get_vault_tvl(atomic);
    let new_utilization = if (vault_tvl == 0) 0 else math::div(new_held_value,
    vault_tvl);
    [...]
}
```

The utilization denominator can also be inflated temporarily. A deposit immediately increases `total_shares`, from which `get_vault_tvl` derives the denominator. An eligible depositor can therefore add capital before a swap so that the post-swap exposure falls below the percentage cap and is evaluated against a lower-utilization fee tier, then remove the corresponding shares. Instant withdrawal reduces `total_shares` immediately, while queued withdrawal reduces it when the request is processed; neither path revalidates outstanding source exposure against the percentage limit after the denominator decreases.

Consequently, the atomic vault may retain source exposure above its configured utilization percentage after the temporary capital exits, leaving the remaining liquidity providers with a greater source concentration than the swap-time check permitted. At zero TVL, a nominal zero-percent utilization cap does not prevent positive exposure when residual or sub-account collateral can fund the swap.

Remediation

Reject a swap whenever `new_held_value` is positive and `vault_tvl` is zero, and explicitly reject positive `new_held_value` whenever `max_utilization_percentage` is zero. Require every whitelisted

© 2026 Otter Audits LLC. All Rights Reserved.

source to have a safely bounded, nonzero `max_held_value_absolute` so exposure remains capped independently of TVL. To preserve percentage-based enforcement, calculate capacity from TVL that cannot be increased and immediately withdrawn, or prevent withdrawal of the shares that supplied the swap's capacity until the corresponding source exposure has been resolved.

Patch

Resolved in 0e27e31.

© 2026 Otter Audits LLC. All Rights Reserved.

Partition-Dependent Tier Fees and Exposure Rounding

ID	Severity	Status
OS-ALV-ADV-07	Low	Ack'd

Description

The swap fee depends on how a user partitions a position across calls. In `do_swap_ember_shares_core`, the protocol derives a single multiplier from the post-swap utilization and applies the resulting fee percentage to the call's entire gross amount. Consequently, one swap that ends in a higher tier pays the higher multiplier across its full gross amount, whereas a sequence reaching the same final exposure can apply lower multipliers to earlier portions before crossing that tier. The fee curve is therefore not additive, allowing split swaps to incur a lower total fee than an equivalent unsplit swap.

The post-swap exposure calculation also converts the existing and incoming share balances separately before adding their collateral values. In `current_held_share_value_in_collateral`, the existing custodied and pending shares are aggregated and then converted with floor rounding. `do_swap_ember_shares_core` separately floor-converts the incoming shares and adds that gross amount to the result. The sum of these independently rounded values can be one collateral unit below the floor-rounded value of the aggregate shares held after the incoming balance is joined.

This understated value is used for the absolute held-value ceiling, the utilization cap, the fee-tier lookup, and the emitted post-swap utilization. A swap may therefore pass a configured exposure boundary despite the subsequently aggregated position being one unit higher, or select a cheaper tier when that unit crosses a threshold. The preview path repeats the same calculation, so it does not correct the discrepancy. As a result, users may pay less for the same final source exposure, while the checked and reported utilization can differ from the aggregate position immediately after the swap.

Remediation

Define the tiered fee as a cumulative marginal cost function and charge each swap the difference between the cumulative cost at the post-swap exposure and the cumulative cost at the pre-swap exposure. Calculate post-exposure by adding raw shares first and converting once. Use that same aggregate, once-rounded value for the absolute cap, utilization cap, tier selection, preview result, and swap event.

Patch

Acknowledged by the team.

© 2026 Otter Audits LLC. All Rights Reserved.

Transient Source TVL Inflates Epoch Swap Capacity

ID	Severity	Status
OS-ALV-ADV-08	Low	Ack'd

Description

If governance intends the 24-hour EmberShare percentage limit to be measured against durable or lagged source-vault TVL, the implementation does not enforce that intended semantic. In `do_swap_ember_shares_core`, the source vault's current TVL is read at swap execution time, without using an epoch-start snapshot, lagged value, or time-weighted value.

sources/atomic_liquidity_vault.move

```
fun do_swap_ember_shares_core<T, R>([...]): (u64, u64, u64, u64, u64) {
    [...]
    let now = clock::timestamp_ms(clock);
    let source_tvl = vault::get_vault_tvl(source_vault);
    [...]
}
```

The same function resets expired epoch accounting, calculates the percentage cap from that current `source_tvl`, and permits the swap when the accumulated gross amount remains within the resulting cap. Under the stated governance intent, a temporary source-vault deposit can therefore increase the permitted epoch volume immediately. The separate `max_held_value_absolute` check limits the post-swap amount held by Atomic when configured, but it does not impose an absolute limit on cumulative volume during the epoch.

sources/atomic_liquidity_vault.move

```
fun do_swap_ember_shares_core<T, R>([...]): (u64, u64, u64, u64, u64) {
    [...]
    {
        [...]
        // Reset the rolling 24h epoch if it has expired (or hasn't started).
        let epoch_used_now = if (cfg.epoch_started_at == 0
            || now >= cfg.epoch_started_at + EMBER_SHARE_EPOCH_DURATION_MS) {
            cfg.epoch_started_at = now;
            cfg.epoch_used_amount = 0;
            0
        } else {
            cfg.epoch_used_amount
        };
        let cap = math::mul(source_tvl, cfg.max_capacity_percentage);
        assert!(epoch_used_now + gross_amount <= cap, EEmberShareCapExceeded);
        cfg.epoch_used_amount = epoch_used_now + gross_amount;
        [...]
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
        assert!(
            cfg.max_held_value_absolute == 0
                || new_held_value <= cfg.max_held_value_absolute,
            EEmberShareHeldValueCapExceeded,
        );
        [...]
    };
    [...]
}
```

Where an account is permitted to deposit into the source vault, it can receive newly minted receipt coins and immediately submit some of them to an Atomic swap. The remaining receipts can subsequently enter the source vault's queued redemption flow. After that withdrawal is processed, Atomic may have acquired more source exposure during the epoch than the intended percentage of the source's durable or post-withdrawal TVL, even though each swap passed the capacity check against current TVL at execution time. This issue is conditional on the percentage limit being intended to represent durable or lagged source TVL and is distinct from utilization checks based on the Atomic vault's TVL.

Remediation

If governance intends the percentage limit to represent durable or lagged source TVL, base it on `min(current_source_tvl, lagged_or_time_weighted_source_tvl)`, with the durable value maintained independently of the swap transaction. Add a mandatory absolute epoch-volume cap.

Patch

Acknowledged by the team.

© 2026 Otter Audits LLC. All Rights Reserved.

Nominal Payout Accounting Ignores Dust Shortfalls

ID	Severity	Status
OS-ALV-ADV-09	Low	✓ Resolved

Description

The audited `pay_tolerating_dust` helper transfers the lesser of the requested amount and the available balance, permits a shortfall of up to 1,000 base units, and returns the amount actually transferred.

In `do_swap_ember_shares_core`, the slippage check compares `min_net_out` with the nominal `immediate_payout` before the transfer. The function then invokes `pay_tolerating_dust` without retaining its return value, records nominal payout data in the swap event, and returns the nominal immediate payout. Consequently, satisfying the nominal check does not ensure that the receiver actually obtains at least `min_net_out`.

sources/atomic_liquidity_vault.move

```
fun do_swap_ember_shares_core<T, R>([...]): (u64, u64, u64, u64, u64) {
    [...]
    // Slippage guard: the immediate collateral actually transferred to the
    // receiver (net, or net + holdback for a holdback-waived receiver) must
    // meet the caller's minimum, protecting against a utilization-driven fee
    // spike between preview and execution.
    assert!(immediate_payout >= min_net_out, ESlippageToleranceExceeded);
    [...]
    pay_tolerating_dust(&mut atomic.balance, immediate_payout, receiver, ctx);
    [...]
}
```

The no-strategy and Suilend swap wrappers return the nominal payout produced by the shared core, so they may report more than the transfer actually delivered.

A successful swap can therefore consume Ember receipt tokens while delivering up to 1,000 base units less than the reported amount. If the nominal immediate payout is no greater than the tolerance, the receiver could obtain zero while the nominal payout still satisfies `min_net_out`. Events and return values can likewise overstate the transferred amount, causing indexers and integrations to record a payment that did not fully occur.

Remediation

Base slippage checks, return values, and payout events on the amount actually transferred rather than the nominal amount.

Patch

Resolved in fab37ee.

© 2026 Otter Audits LLC. All Rights Reserved.

Missing Version Checks in Shared-Config Mutators

ID	Severity	Status
OS-ALV-ADV-10	Low	✓ Resolved

Description

`guardian_pause_non_admin_operations` verifies that the caller is the configured guardian and that the requested pause state differs from the current state before updating the shared `ProtocolConfig`. As shown in the function, it does not invoke `verify_supported_package`, which elsewhere requires the configuration version to equal the package's `VERSION` constant.

sources/admin.move

```
199 public fun guardian_pause_non_admin_operations(
200     config: &mut ProtocolConfig,
201     pause: bool,
202     ctx: &TxContext,
203 ) {
204     verify_guardian(config, ctx);
205     assert!(pause != config.pause_non_admin_operations, ESameValue);
206     config.pause_non_admin_operations = pause;
207     events::emit_pause_non_admin_operations_event(pause);
208 }
```

The supported-version mechanism is intended to retire older package semantics by advancing the version stored in the shared configuration from the newly deployed package. Because the guardian endpoint and several adjacent configuration mutators omit this guard, an older package can continue operating on the newly versioned configuration instead of aborting as unsupported. Consequently, a guardian that remains configured after an upgrade can invoke the old endpoint to pause or unpause non-admin operations, bypassing restrictions implemented only in the newer bytecode.

Remediation

Call `verify_supported_package(config)` at the beginning of every public endpoint that reads or mutates the shared `ProtocolConfig`, including guardian and administrator configuration paths. Exempt only the narrowly scoped version-increase or migration endpoint that must accept the previous configuration version.

Patch

Resolved in fab37ee.

© 2026 Otter Audits LLC. All Rights Reserved.

Full Suilend Redemption Can Block Strategy Exit

ID	Severity	Status
OS-ALV-ADV-11	Low	Ack'd

Description

`clear_suilend_state` removes the vault's stored cToken balance and, when that balance is nonzero, submits the entire amount to Suilend's combined redemption and liquidity-withdrawal helper. The function does not limit the redemption amount or preflight the market conditions that govern whether the full balance can be redeemed, as shown in the referenced snippet.

sources/strategy.move

```
public(package) fun clear_suilend_state<P, T>(
    uid: &mut UID,
    market: &mut LendingMarket<P>,
    clock: &Clock,
    ctx: &mut TxContext,
): Balance<T> {
    let mut drained = balance::zero<T>();
    if (dynamic_field::exists_with_type<vector<u8>, Balance<CToken<P, T>>>(uid,
    KEY_SUILEND_CTOKENS)) {
        let bal: Balance<CToken<P, T>> = dynamic_field::remove(uid,
        KEY_SUILEND_CTOKENS);
        let held = balance::value(&bal);
        if (held > 0) {
            let reserve_index = *dynamic_field::borrow<vector<u8>, u64>(
                uid, KEY_SUILEND_RESERVE_INDEX,
            );
            let received = lending_market::redeem_ctokens_and_withdraw_liquidity<P, T>(
                market,
                reserve_index,
                clock,
                coin::from_balance(bal, ctx),
                option::none(),
                ctx,
            );
            balance::join(&mut drained, coin::into_balance(received));
        }[...]
    };[...]
}
```

This all-or-nothing operation can conflict with the pinned Suilend implementation's requirement that a redemption leave at least 100 underlying units of available liquidity and at least 100 cToken units in global supply. A full redemption by the sole or final holder would reduce these quantities

© 2026 Otter Audits LLC. All Rights Reserved.

below their required floors. The same call can also exceed the market-wide remaining outflow allowance when the vault holds a sufficiently large position, causing the redemption to abort.

An abort reverts the removal of the cToken balance, so the vault's assets do not disappear. The caller also joins the redeemed assets and changes the strategy tag only after `clear_suilend_state` returns successfully. Nevertheless, the strategy cannot be cleared atomically under these conditions, which may delay an emergency evacuation and require a non-vault holder to maintain the minimum seed or the operator to unwind the position over multiple transactions and limiter windows.

Remediation

Before attempting a Suilend redemption, inspect the reserve's available underlying liquidity, global cToken supply, and remaining market-wide outflow capacity. Cap each redemption so that it preserves the required 100-unit liquidity and cToken-supply floors and does not exceed either physical liquidity or the current limiter allowance. Require a non-vault minimum seed when activating the strategy, and provide a staged exit path that retains the cToken balance, reserve index, and Suilend strategy tag until the committed cToken balance is exactly zero. Document the required external seed and the procedure for continuing an unwind after the limiter window recovers.

Patch

Acknowledged by the team.

© 2026 Otter Audits LLC. All Rights Reserved.

Annual Fee Undercollection from a Shared Fee Ledger

ID	Severity	Status
OS-ALV-ADV-12	Low	Ack'd

Description

The Atomic vault records annual platform fees, instant-withdraw fees, and swap fees in the same `fee.accrued` field. As shown in `charge_accrued_platform_fees`, each annual accrual derives TVL from the vault's outstanding shares and then subtracts the entire accrued balance before computing the next fee. This calculation implicitly treats every accrued fee as value already included in the share-derived TVL, although the other fee sources do not have that accounting meaning.

sources/atomic_liquidity_vault.move

```
fun charge_accrued_platform_fees<T>(vault: &mut AtomicLiquidityVault<T>, clock:
&Clock) {
    [...]
    // Compute fees against tvl net of already-accrued fees so the protocol
    // doesn't re-tax the pending-fee portion of TVL on each accrual round.
    // Mirrors EVM AtomicLiquidity._chargeAccruedPlatformFees' tvlForFeeCalc.
    // TVL is read as u128 so this exit-path accrual can't abort on an
    // over-u64 TVL (which would lock every exit — see get_vault_tvl_u128).
    let tvl = get_vault_tvl_u128(vault);
    let accrued = (vault.fee.accrued as u128);
    let tvl_for_fee_calc = if (tvl > accrued) tvl - accrued else 0;
    let fee_amount = common::compute_platform_fee_delta(
        tvl_for_fee_calc,
        vault.fee_percentage,
        last_charged_at,
        current_time,
    );
    [...]
}
```

In `do_instant_withdraw_core`, annual fees are settled before the withdrawal, after which the function calculates the gross withdrawal, burns the corresponding shares, and adds the instant-withdraw fee to `fee.accrued`. Burning the shares already reduces share-derived TVL by the full gross withdrawal, while only the net amount is transferred from the vault. Consequently, subtracting the retained instant-withdraw fee from that reduced TVL during the next annual accrual excludes the fee value from the annual base a second time.

sources/atomic_liquidity_vault.move

```
fun do_instant_withdraw_core<T>(
    vault: &mut AtomicLiquidityVault<T>,
    config: &ProtocolConfig,
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
    shares: u64,
    receiver: address,
    clock: &Clock,
    ctx: &TxContext,
): (u64, u64, u64) {
    [...]
    // Burn user shares + advance cumulative.
    decrement_account_shares(&mut vault.shares, owner, shares);
    vault.total_shares = vault.total_shares - shares;
    vault.cumulative_instant_withdraw_shares = new_cumulative;
    // Accrue the fee (collected later via collect_platform_fee).
    if (fee_amount > 0) vault.fee.accrued = vault.fee.accrued + fee_amount;
    (gross_amount, fee_amount, net_amount)
}
```

The same mismatch occurs in `do_swap_ember_shares_core`. A swap does not burn Atomic vault shares, but the vault receives the source-vault shares, pays less collateral than their gross value because of the fee, and credits that fee to the common accrued balance. The retained value therefore backs a separate protocol-fee claim without increasing the Atomic vault's share-derived TVL, yet the subsequent annual calculation still subtracts it from that TVL.

sources/atomic_liquidity_vault.move

```
fun do_swap_ember_shares_core<T, R>(
    atomic: &mut AtomicLiquidityVault<T>,
    config: &ProtocolConfig,
    source_vault: &Vault<T, R>,
    share_coin: Coin<R>,
    receiver: address,
    min_net_out: u64,
    clock: &Clock,
    ctx: &mut TxContext,
): (u64, u64, u64, u64, u64) {    [...]
    // full amount as before). A larger gap aborts with `EInsufficientBalance`.
    pay_tolerating_dust(&mut atomic.balance, immediate_payout, receiver, ctx);
    if (fee_amount > 0) atomic.fee.accrued = atomic.fee.accrued + fee_amount;
    [...]
}
```

This treatment is inconsistent with `solvency_margin`, which models accrued fees as an additional claim alongside share-derived TVL, and with fee collection, which settles the combined accrued balance without preserving its origin. Annual platform fees may therefore be understated after instant withdrawals or swaps, causing protocol revenue to depend on the composition of the shared fee ledger rather than solely on the intended annual-fee base.

© 2026 Otter Audits LLC. All Rights Reserved.

Remediation

Track annual platform fees, instant-withdraw fees, and swap fees in separate accounting fields. Define whether previously accrued annual fees should compound, and when calculating `tvl_for_fee_calc`, subtract only the annual-fee component that is mathematically included in the selected TVL base.

Patch

Acknowledged by the team.

© 2026 Otter Audits LLC. All Rights Reserved.

Overflow in Platform Fee Accrual

ID	Severity	Status
OS-ALV-ADV-13	Low	✓ Resolved

Description

The shared `compute_platform_fee_delta` helper calculates an annualized platform fee by multiplying the vault TVL, fee percentage, and elapsed time as `u128` values before applying `FEE_DENOMINATOR`. This ordering allows the intermediate numerator to overflow even when division would produce an economically meaningful result that fits within the returned `u64`.

sources/common.move

```
public(package) fun compute_platform_fee_delta(
    tvl: u128,
    fee_percentage: u64,
    last_charged_at: u64,
    current_time: u64,
): u64 {
    if (current_time <= last_charged_at || fee_percentage == 0 || tvl == 0) {
        return 0
    };
    let elapsed_time_ms = current_time - last_charged_at;
    let numerator = tvl * (fee_percentage as u128) * (elapsed_time_ms as u128);
    math::safe_cast_to_u64(numerator / FEE_DENOMINATOR)
}
```

Both regular and atomic vault fee-checkpoint helpers calculate TVL as a `u128`, subtract previously accrued fees, and pass the result to `compute_platform_fee_delta`. They advance `last_charged_at` only after that call returns. Consequently, an overflowing multiplication aborts the checkpoint without advancing its timestamp, so subsequent checkpoints evaluate the same interval with an equal or larger elapsed time.

Fee checkpointing occurs in regular-vault deposits, share minting, withdrawal settlement, rate and fee updates, and fee collection. Atomic-vault deposits, withdrawal settlement, fee and rate updates, fee collection, instant withdrawals, receipt-share swaps, and holdback release also reach the same helper. Once the intermediate exceeds the `u128` range, these operations can repeatedly abort until the relevant state is changed through a path that does not perform the overflowing calculation.

Remediation

Perform fee-accrual arithmetic at sufficient width and narrow the result only after division and bounds checking.

Patch

Resolved in fab37ee.

© 2026 Otter Audits LLC. All Rights Reserved.

5. Suggestions

Here, we present a discussion of the 14 suggestions we identified during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Severity	Status	Description
OS-ALV-SUG-00	Informational	Ack'd	Stale Share Rates Misallocate Vault Backing
OS-ALV-SUG-01	Informational	Ack'd	Stale Source Rates Shift Losses to Atomic LPs
OS-ALV-SUG-02	Informational	Ack'd	Account-Wide Timestamp Distorts Time-Based Withdrawal Fees
OS-ALV-SUG-03	Informational	Resolved	Incomplete Validation of Initial Vault Configuration
OS-ALV-SUG-04	Informational	Ack'd	Linear Account Lists Increase Operational Gas Costs
OS-ALV-SUG-05	Informational	Resolved	Misleading Net Amount for Holdback-Waived Swaps
OS-ALV-SUG-06	Informational	Resolved	Stale Sequence on Swap Auto-Pull Events
OS-ALV-SUG-07	Informational	Ack'd	Floor-Rounded TVL Weakens the Maximum Cap
OS-ALV-SUG-08	Informational	Resolved	Overflow-Prone Solvency Margin View
OS-ALV-SUG-09	Informational	Resolved	Redundant Source Identifier in Held-Value Helper
OS-ALV-SUG-10	Informational	Resolved	Suilend Market Identity Not Committed at Activation
OS-ALV-SUG-11	Informational	Ack'd	Unauthenticated Sub-Account Attribution for Vault Deposits
OS-ALV-SUG-12	Informational	Ack'd	Missing Witness for Atomic Vault Sharing
OS-ALV-SUG-13	Informational	Resolved	Backing Queries May Abort or Mislead

© 2026 Otter Audits LLC. All Rights Reserved.

Stale Share Rates Misallocate Vault Backing

ID	Severity	Status
OS-ALV-SUG-00	Informational	Ack'd

Description

The atomic vault relies on a manually maintained share rate to value deposits and withdrawals. In `update_vault_rate`, an update is subject to a minimum interval, global rate bounds, and a maximum change, and an update to the existing value is rejected. However, the rate has no maximum age or valuation nonce that user operations can verify, so the stored value may remain usable after the vault's backing changes.

The stale-rate risk directly affects instant withdrawals. As shown in `do_instant_withdraw_core`, the gross collateral amount is calculated from the shares and the stored rate before the shares are burned, without checking when the backing was last valued. If a realized strategy loss or other off-vault shortfall is known before the rate is corrected, an early exiter may receive more than their post-loss pro-rata claim, leaving the shortfall with the remaining shareholders.

sources/atomic_liquidity_vault.move

```
fun do_instant_withdraw_core<T>(
    vault: &mut AtomicLiquidityVault<T>,
    config: &ProtocolConfig,
    shares: u64,
    receiver: address,
    clock: &Clock,
    ctx: &TxContext,
): (u64, u64, u64) {
    [...]
    let gross_amount = math::div(shares, vault.rate.value);
    assert!(gross_amount > 0, EZeroAmount);
    let fee_amount = math::mul(gross_amount, vault.instant_withdraw_fee_percentage);
    let net_amount = gross_amount - fee_amount;
    // Burn user shares + advance cumulative.
    decrement_account_shares(&mut vault.shares, owner, shares);
    vault.total_shares = vault.total_shares - shares;
    vault.cumulative_instant_withdraw_shares = new_cumulative;
    [...]
}
```

The inverse value transfer can occur when backing increases between rate updates. Within `process_request`, request settlement calculates the withdrawal amount from the stored rate, burns the requested shares, deducts configured withdrawal fees, and transfers only the net amount. The deducted fees remain in `vault.balance`, while the settlement does not update or invalidate the stored rate. Deposits remain available during this interval. As shown in `deposit`, deposited collateral is joined to `vault.balance` and shares are minted using the stored rate, without

© 2026 Otter Audits LLC. All Rights Reserved.

a rate-freshness or valuation-state check. A depositor entering after withdrawal fees, returned yield, or other backing have accrued but before the rate is updated may therefore receive shares at the previous conversion and capture part of the value attributable to existing shareholders.

sources/atomic_liquidity_vault.move

```
public fun deposit<T>(
    vault: &mut AtomicLiquidityVault<T>,
    config: &ProtocolConfig,
    coin: Coin<T>,
    min_shares: u64,
    receiver: address,
    clock: &Clock,
    ctx: &mut TxContext,
): u64 {
    [...]
    balance::join(&mut vault.balance, coin::into_balance(coin));
    let shares_minted = math::mul(amount, vault.rate.value);
    assert!(shares_minted > 0, EZeroAmount);
    assert!(shares_minted >= min_shares, ESlippageToleranceExceeded);
    [...]
}
```

Because the same stored rate governs both entry and exit, backing changes that occur between manual updates can redistribute value depending on transaction timing. Early exiters may avoid realized losses, while late depositors may share in value accumulated before their deposits; in either case, the incumbent cohort retains less value than its updated pro-rata ownership would imply.

Remediation

Bind rate updates to a valuation nonce and provide a same-value heartbeat so freshness does not require changing the rate. Introduce an immediately settable invalid or pending-valuation state, and allow verified loss corrections without waiting for the ordinary interval. Credit retained fees or returned yield through a cumulative per-share index.

Patch

Acknowledged by the team.

© 2026 Otter Audits LLC. All Rights Reserved.

Stale Source Rates Shift Losses to Atomic LPs

ID	Severity	Status
OS-ALV-SUG-01	Informational	Ack'd

Description

The `do_swap_ember_shares_core` function values the supplied source-vault shares using the source vault's current rate, but does not verify when that rate was last updated or otherwise require a fresh valuation. It uses this valuation to calculate the fee and optional holdback, transfers the immediate collateral payout, and retains the source shares for later redemption. Because the holdback may be zero or waived for the account, the atomic vault can advance nearly the entire swap-time valuation before receiving any source-vault proceeds.

The retained shares are subsequently submitted to the source vault's withdrawal queue. In `process_request`, the source vault calculates `withdraw_amount` from the rate present when the queued request is processed, rather than preserving the rate used by the atomic swap. An adverse rate correction between these operations can therefore cause the atomic vault to recover less collateral than the valuation on which its earlier payout was based.

A holder can sell source shares before such a correction and transfer the valuation difference to the atomic vault. When the immediate payout exceeds the eventual source redemption proceeds, the shortfall reduces the collateral backing atomic-vault LPs. Swap fees and configurable holdbacks mitigate the difference only when they are sufficiently large. They do not establish a bound on the source rate's age or guarantee that the advance remains below the eventual recovery. The resulting deficit may also leave part of the recorded swap-fee accrual without corresponding collateral.

Remediation

Reject source-share swaps unless the source valuation is within an explicitly configured freshness window. Because `update_vault_rate` rejects an unchanged value, introduce a valuation nonce or same-value heartbeat that updates the freshness marker without requiring the reported rate to change. Require a non-waivable minimum holdback sufficient to cover the maximum adverse rate movement permitted during the redemption window, and keep that holdback associated with the source redemption and locked until the corresponding collateral has been received by the atomic vault.

Patch

Acknowledged by the team.

© 2026 Otter Audits LLC. All Rights Reserved.

Account-Wide Timestamp Distorts Time-Based Withdrawal Fees

ID	Severity	Status
OS-ALV-SUG-02	Informational	Ack'd

Description

The atomic vault represents all of an account's deposit lots with one amount-weighted timestamp. The deposit path accepts a caller-chosen receiver, and the timestamp update weights the receiver's active shares together with shares already queued for withdrawal. Consequently, a deposit made for another account can alter the fee age used for both its active position and its pending requests.

This weighting limits the effect of very small deposits but does not preserve fee eligibility across the threshold's binary boundary. If a pending balance `B` is aged only `epsilon` beyond a threshold `D`, a third-party deposit greater than `B * epsilon / D` moves the weighted timestamp back inside the fee window. Conversely, for old shares `O` aged `A` and fresh shares `N`, the resulting apparent age is `O * A / (O + N)`. Whenever this remains above the threshold, the fresh shares inherit fee-free status despite the source comment's stated objective that new shares should not inherit the old deposit's age.

The queued `WithdrawalRequest` records the owner, receiver, shares, estimated withdrawal amount, request timestamp, and sequence number, but does not snapshot the applicable time-based fee state. During settlement, `process_request` instead reads the owner's current `last_deposit_ts` and makes one binary eligibility decision for the request. If eligible, it calculates the time-based fee against the request's entire gross withdrawal amount rather than separating expired and unexpired deposit lots.

A permitted third-party depositor may therefore re-age an already queued withdrawal and cause its owner to receive less than anticipated when the request is processed. The same account-wide timestamp may also allow an aged balance to make a larger fresh deposit immediately eligible for withdrawal without the intended time-based fee. Fees withheld during processing remain in the vault balance after the request shares are burned, so the retained value benefits the remaining shares rather than being transferred directly to the depositor who changed the timestamp.

Remediation

Restrict `deposit` to `receiver == ctx.sender()` unless the receiver has explicitly authorized the third-party deposit. When `redeem_shares` queues a withdrawal, store the fee eligibility or immutable fee-age state that settlement must use so later deposits cannot change the request's treatment. To prevent both re-aging and inherited maturity comprehensively, track shares by deposit lot or expiry bucket and charge the time-based fee only against the portion whose fee window has not expired.

Patch

Acknowledged by the team.

© 2026 Otter Audits LLC. All Rights Reserved.

Incomplete Validation of Initial Vault Configuration

ID	Severity	Status
OS-ALV-SUG-03	Informational	Resolved

Description

The classic `create_vault` constructor accepts configuration values that its administrative setters would reject. As shown in `create_vault`, construction checks the rate-update interval, minimum withdrawal amount, receipt-token supply, role separation, role collisions with the sub-account list, and fee percentage, but it stores `max_tvl` and `max_rate_change_per_update` without requiring a positive maximum TVL or constraining the permitted rate change to the setter's valid interval. Consequently, an `AdminCap` holder can create a vault with a zero TVL limit or an excessive rate-change allowance.

sources/vault.move

```
public fun create_vault<T,R>([...]): Vault<T,R> {
    admin::verify_supported_package(config);
    assert!(rate_update_interval >= admin::get_min_rate_interval(config) &&
    rate_update_interval <= admin::get_max_rate_interval(config), EInvalidInterval);
    assert!(min_withdrawal_shares > 0, EInvalidAmount);
    assert!(coin::total_supply<R>(&treasury_cap) == 0,
    EReceiptTokenTreasuryCapNotEmpty);
    assert!(
        admin != @0 &&
        operator != @0 &&
        admin != operator &&
        !vector::contains(&sub_accounts, &admin) &&
        !vector::contains(&sub_accounts, &operator),
        EInvalidAccount
    );
    assert!(fee_percentage <= admin::get_max_allowed_fee_percentage(config),
    EInvalidFeePercentage);
    [...]
}
```

The `create_atomic_liquidity_vault` constructor does enforce a positive maximum TVL and bounds `max_rate_change_per_update`. However, its sub-account validation only excludes collisions with the admin, operator, and rate manager. It does not reject `@0` or repeated addresses. The classic constructor has the same omission for its sub-account vector.

sources/atomic_liquidity_vault.move

```
public fun create_atomic_liquidity_vault<T>([...]): AtomicLiquidityVault<T> {
    [...]
    assert!(max_tvl > 0, EInvalidAmount);
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
    assert!(max_rate_change_per_update > 0 && max_rate_change_per_update < PERCENT_MAX,
    EInvalidAmount);
    assert!(
        admin != @0
            && operator != @0
            && rate_manager != @0
            && admin != operator
            && admin != rate_manager
            && operator != rate_manager
            && !vector::contains(&sub_accounts, &admin)
            && !vector::contains(&sub_accounts, &operator)
            && !vector::contains(&sub_accounts, &rate_manager),
        EInvalidAccount,
    );
    [...]
}
```

Atomic vault constructor validation

These constructor checks are inconsistent with the later configuration paths. Both vault types reject `@0` when adding or removing an individual list entry and reject duplicate additions, while the maximum-TVL and maximum-rate-change setters enforce the bounds omitted by classic vault creation. An initially allowlisted zero address therefore cannot be removed through the ordinary sub-account setter, and removing a duplicated address once leaves another copy in the list. Because the operator transfer paths authorize destinations by membership in this list, an operator can transfer vault assets to an allowlisted `@0` address, potentially orphaning them. A zero maximum TVL can prevent positive deposits, while an excessive maximum-rate-change setting weakens the intended limit on subsequent rate updates.

Remediation

Apply the same value bounds, role constraints, and nonzero and unique sub-account invariants during vault creation as in later configuration updates.

Patch

Resolved in dc3a25a, 75547f6, and 98a0ab8.

© 2026 Otter Audits LLC. All Rights Reserved.

Linear Account Lists Increase Operational Gas Costs

ID	Severity	Status
OS-ALV-SUG-04	Informational	Ack'd

Description

Blacklist, sub-account, deposit-allowlist, fee-exemption, waiver, and active-source membership are stored in vectors or `VecSet` collections whose lookup and removal costs grow with collection length. The shared `update_accounts` helper first searches the entire vector for the target address and, when removing an entry, removes it by its discovered index, as shown in the below snippet.

sources/common.move

```
public(package) fun update_accounts(
    accounts: &mut vector<address>,
    account: address,
    status: bool,
) {
    assert!(account != @0, EZeroAddress);
    let (exists, index) = vector::index_of(accounts, &account);
    if (status) {
        assert!(!exists, EAlreadyExists);
        vector::push_back(accounts, account);
    } else {
        assert!(exists, ENotFound);
        vector::remove(accounts, index);
    };
}
```

These collections are consulted on user-facing paths. In particular, `deposit` checks the depositor and receiver against both the blacklist and sub-account vectors, then checks the depositor against the optional deposit allowlist, as shown in the second snippet. Consequently, deposit authorization becomes progressively more expensive as these lists grow. Related redemption and withdrawal-processing paths also perform vector membership checks.

sources/atomic_liquidity_vault.move

```
public fun deposit<T>([...]): u64 {
    [...]
    let depositor = ctx.sender();
    assert!(!vector::contains(&vault.blacklisted, &depositor), EBlacklistedAccount);
    assert!(!vector::contains(&vault.blacklisted, &receiver), EBlacklistedAccount);
    assert!(!vector::contains(&vault.sub_accounts, &depositor), ESubAccount);
    assert!(!vector::contains(&vault.sub_accounts, &receiver), ESubAccount);
    // If a deposit allowlist exists, only listed addresses may deposit.
    // Matches `vault::deposit_asset_v2` — gate is keyed by the sender
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
        // (the party funding the deposit), not the receiver. Withdrawals
        // remain ungated so users who received shares before being de-listed
        // can still redeem them.
        if (dynamic_field::exists_with_type<vector<u8>, vector<address>>(&vault.id,
        DEPOSIT_USER_LIST_DYNAMIC_FIELD)) {
            let list = dynamic_field::borrow<vector<u8>, vector<address>>(
                &vault.id, DEPOSIT_USER_LIST_DYNAMIC_FIELD,
            );
            assert!(vector::contains(list, &depositor), EDepositNotAllowed);
        };
        [...]
    }
```

Administrative mutations compound this cost by copying and emitting complete list snapshots. Both `set_sub_account` and `set_blacklisted_account` retain the previous vector, mutate the current vector through `update_accounts`, and pass the complete previous and current vectors to their update events. Thus, even a one-address change incurs work and event data proportional to the list size. Sufficiently large collections may increase the cost of routine deposits and withdrawals and may cause urgent blacklist or role changes to approach transaction limits precisely when they are needed.

Remediation

Use address-keyed dynamic fields or tables for membership and impose explicit list-size bounds. Emit events containing only the affected address, action, and configuration version.

Patch

Acknowledged by the team.

© 2026 Otter Audits LLC. All Rights Reserved.

Misleading Net Amount for Holdback-Waived Swaps

ID	Severity	Status
OS-ALV-SUG-05	Informational	✓ Resolved

Description

The swap calculation subtracts both the fee and the configured raw holdback from `gross_amount` when deriving `net_amount`, regardless of whether the owner is exempt from holdback. In `do_swap_ember_shares_core`, a holdback-waived owner instead receives `net_amount` plus the raw holdback as the immediate payout, while `queued_holdback` is set to zero.

sources/atomic_liquidity_vault.move

```
fun do_swap_ember_shares_core<T, R>([...]): (u64, u64, u64, u64, u64) {
    [...]
    // Pay immediate collateral. If the user is holdback-waived they
    // receive net + the would-be-held portion in one go.
    let immediate_payout = if (holdback_waived) {
        net_amount + holdback_amount_raw
    } else {
        net_amount
    };
    // Slippage guard: the immediate collateral actually transferred to the
    // receiver (net, or net + holdback for a holdback-waived receiver) must
    // meet the caller's minimum, protecting against a utilization-driven fee
    // spike between preview and execution.
    assert!(immediate_payout >= min_net_out, ESlippageToleranceExceeded);
    let queued_holdback = if (holdback_waived) 0 else holdback_amount_raw;
    [...]
}
```

The same function transfers the immediate payout but calls `emit_ember_share_swapped_event` with `queued_holdback` and the unchanged, pre-waiver `net_amount`. Consequently, for a holdback-waived swap, the event reports no queued holdback while its field named `net_amount` remains lower than the collateral paid to the receiver. An event consumer that treats `net_amount` as the actual immediate transfer may therefore undercount the vault's immediate collateral outflow. The correct payout remains reconstructible as `gross_amount` minus `fee_amount` when the reported holdback is zero, but this requires consumers to disregard the apparent meaning of `net_amount` and infer the waiver outcome from the other fields.

Remediation

Report the actual payout and encode computed, queued, and waived holdback amounts with unambiguous event fields.

Patch

Resolved in dc3a25a.

© 2026 Otter Audits LLC. All Rights Reserved.

Stale Sequence on Swap Auto-Pull Events

ID	Severity	Status
OS-ALV-SUG-06	Informational	✓ Resolved

Description

The Suilend-backed swap wrapper calculates the required payout and calls `ensure_liquidity_suilend` before invoking `do_swap_ember_shares_core`. When an auto-pull occurs, the helper emits a `StrategyWithdrawnEvent` using the vault's current `sequence_number`. It neither reserves a new sequence nor receives the sequence assigned to the pending swap.

The swap core increments the sequence only later, after processing the swap, and uses that new value for the queued holdback entry and `EmberShareSwappedEvent`. The strategy-withdrawal event can therefore retain the sequence of the preceding vault action rather than the sequence of the swap that caused it. An off-chain consumer that associates related events solely through `(vault, sequence_number)` may attribute the withdrawal to the preceding action and fail to correlate it with its actual parent swap.

Remediation

Reserve one action sequence before emitting a group of related events and reuse it across the strategy withdrawal and its parent action.

Patch

Resolved in c54e65a and 75547f6.

© 2026 Otter Audits LLC. All Rights Reserved.

Floor-Rounded TVL Weakens the Maximum Cap

ID	Severity	Status
OS-ALV-SUG-07	Informational	Ack'd

Description

The maximum-TVL limit is enforced using an integer-valued representation of the vault's share claim. In `update_vault_max_tvl`, the administrator may set a new cap whenever `get_vault_tvl_u128` returns a value less than or equal to that cap. Consequently, this check can accept a cap that is slightly below the exact rational value represented by the outstanding shares.

sources/atomic_liquidity_vault.move

```
public fun update_vault_max_tvl<T>(
    vault: &mut AtomicLiquidityVault<T>,
    config: &ProtocolConfig,
    max_tvl: u64,
    ctx: &TxContext,
) {
    admin::verify_supported_package(config);
    assert_admin(vault, ctx);
    assert!(max_tvl > 0, EInvalidAmount);
    assert!(max_tvl != vault.max_tvl, ESameValue);
    assert!(get_vault_tvl_u128(vault) <= (max_tvl as u128), EMaxTVLReached);
    let previous = vault.max_tvl;
    vault.max_tvl = max_tvl;
    let seq = bump_sequence(vault);
    [...]
}
```

The discrepancy originates in `get_vault_tvl_u128`, which converts `total_shares` into collateral units using fixed-point integer division. Because `math::div_u128` rounds the quotient down, any fractional remainder is discarded. An exact share claim in the interval between `max_tvl` and `max_tvl + 1` is therefore reported as `max_tvl`.

sources/atomic_liquidity_vault.move

```
2566 public fun get_vault_tvl_u128<T>(vault: &AtomicLiquidityVault<T>): u128 {
2567     if (vault.total_shares == 0) 0
2568     else math::div_u128(vault.total_shares, vault.rate.value)
2569 }
```

The deposit flow applies the same floor-rounded TVL check after increasing `total_shares`. Deposits may therefore leave the exact rational share claim slightly above the configured maximum while satisfying the on-chain comparison. The excess is limited to less than one unit at the precision returned by the TVL calculation, but the configured value is not a mathematically strict

© 2026 Otter Audits LLC. All Rights Reserved.

upper bound. Integrations or invariants that assume exact exposure never exceeds `max_tvl` may consequently be inaccurate by this rounding amount.

Remediation

If `max_tvl` is intended to be a strict upper bound, use an enforcement-specific ceiling calculation for both `update_vault_max_tvl` and the post-deposit cap check, while retaining floor division for accounting or display functions that require it.

Patch

Acknowledged by the team.

© 2026 Otter Audits LLC. All Rights Reserved.

Overflow-Prone Solvency Margin View

ID	Severity	Status
OS-ALV-SUG-08	Informational	Resolved

Description

The informational `solvency_margin` view can abort instead of returning a solvency result for sufficiently large values. As shown below, the function obtains the idle collateral backing as a `u64`, computes the modeled claims by adding the share-derived TVL to accrued platform fees, and returns both values with the `backing >= claims` status.

sources/atomic_liquidity_vault.move

```
2610  public fun solvency_margin<T>(
2611      vault: &AtomicLiquidityVault<T>,
2612  ): (u64, u64, bool) {
2613      let backing = balance::value(&vault.balance);
2614      let claims = get_vault_tvl(vault) + vault.fee.accrued;
2615      (backing, claims, backing >= claims)
2616  }
```

`get_vault_tvl` performs its fixed-point calculation using a `u128` intermediate but narrows the result to `u64`, aborting if the result exceeds `u64::MAX`. Even when that conversion succeeds, the subsequent addition of the two `u64` claim components can overflow if their sum exceeds `u64::MAX`. Because this function is read-only, an abort does not directly modify vault state. However, it may prevent off-chain monitors from obtaining solvency status precisely when the vault is oversized or potentially underbacked, removing the signal the view is intended to provide.

Remediation

Keep solvency calculations and outputs at a width sufficient for the maximum modeled values, avoiding lossy narrowing before comparison.

Patch

Resolved in dc3a25a.

© 2026 Otter Audits LLC. All Rights Reserved.

Redundant Source Identifier in Held-Value Helper

ID	Severity	Status
OS-ALV-SUG-09	Informational	Resolved

Description

The private `current_held_share_value_in_collateral` helper accepts both a typed `source_vault` reference and a separate `source_id`, even though the identifier can be derived from the vault object. As shown in the helper, `source_id` selects the custodied receipt-share balance and controls whether pending withdrawals are queried, while `source_vault` supplies the pending-share state and conversion rate. These independent inputs therefore permit the helper to combine state associated with different source identities if an internal caller passes a mismatched pair.

sources/atomic_liquidity_vault.move

```
fun current_held_share_value_in_collateral<T, R>(
    atomic: &AtomicLiquidityVault<T>,
    source_vault: &Vault<T, R>,
    source_id: ID,
): u64 {
    let key = CollectedEmberSharesKey { source_vault_id: source_id };
    [...]
    let pending_shares = if (!vec_set::is_empty(&atomic.redemption_active_sources)
        && vec_set::contains(&atomic.redemption_active_sources, &source_id)) {
        let atomic_addr = object::uid_to_address(&atomic.id);
        vault::get_account_total_pending_withdrawal_shares<T, R>(
            source_vault, atomic_addr,
        )
    } else { 0 };
    [...]
}
```

The current production call sites derive `source_id` from the same `source_vault`, and the helper is private, so this does not result in an issue. However, a future internal call may inadvertently supply an inconsistent identifier, causing held exposure or utilization calculations to incorporate custody or redemption-tracking state under one source identity while using another vault's state and conversion rate. This may produce an incorrect risk view for the affected source.

Remediation

Derive each source identifier from the typed source-vault object instead of accepting an independently supplied identifier.

Patch

Resolved in fab37ee.

© 2026 Otter Audits LLC. All Rights Reserved.

Suilend Market Identity Not Committed at Activation

ID	Severity	Status
OS-ALV-SUG-10	Informational	✓ Resolved

Description

The admin activates Suilend through `set_strategy_suilend<T>` by providing only a numeric reserve index. The function does not receive the intended `LendingMarket<P>` or otherwise bind the strategy configuration to a market object or the defining type `P`.

sources/atomic_liquidity_vault.move

```
public fun set_strategy_suilend<T>(
    vault: &mut AtomicLiquidityVault<T>,
    config: &ProtocolConfig,
    reserve_array_index: u64,
    ctx: &TxContext,
) {
    admin::verify_supported_package(config);
    assert_admin(vault, ctx);
    assert!(vault.strategy_kind == strategy::strategy_none(), EStrategyMismatch)
    strategy::init_suilend_state(&mut vault.id, reserve_array_index)
    let previous = vault.strategy_kind;
    vault.strategy_kind = strategy::strategy_suilend();
    [...]
}
```

The underlying `strategy::init_suilend_state` implementation persists only `reserve_array_index` as a dynamic field. Consequently, activation does not establish on-chain which Suilend market and market type the index is intended to identify.

sources/strategy.move

```
public(package) fun init_suilend_state(uid: &mut UID, reserve_array_index: u64) {
    assert!(!dynamic_field::exists_with_type<vector<u8>, u64>(uid,
    KEY_SUILEND_RESERVE_INDEX), ESuilendStateAlreadyExists);
    dynamic_field::add(uid, KEY_SUILEND_RESERVE_INDEX, reserve_array_index);
}
```

The market identity is instead selected operationally when the trusted operator first calls `supply_to_suilend_strategy<T, P>`. That call accepts a generic `P` and a caller-provided `LendingMarket<P>`, then delegates the deposit to the stored reserve index. The delegated strategy function creates or updates the correspondingly typed `Balance<CToken<P, T>>` slot without comparing the supplied market against an activation-time commitment.

© 2026 Otter Audits LLC. All Rights Reserved.

This missing commitment also affects cleanup. As shown in `clear_suilend_state<P, T>`, the function checks for the cToken balance using the caller-selected `P`. If that type does not match the existing `Balance<CToken<P, T>>` dynamic field, the check evaluates to false. The function may then remove the reserve-index metadata and return a zero balance without redeeming the actual typed cToken slot.

sources/strategy.move

```
public(package) fun clear_suilend_state<P, T>(
    uid: &mut UID,
    market: &mut LendingMarket<P>,
    clock: &Clock,
    ctx: &mut TxContext,
): Balance<T> {
    let mut drained = balance::zero<T>();
    if (dynamic_field::exists_with_type<vector<u8>, Balance<CToken<P, T>>>(uid,
    KEY_SUILEND_CTOKENS)) {
        [...]
    };
    if (dynamic_field::exists_with_type<vector<u8>, u64>(uid,
    KEY_SUILEND_RESERVE_INDEX)) {
        let _: u64 = dynamic_field::remove(uid, KEY_SUILEND_RESERVE_INDEX);
    };
    drained
}
```

The same ambiguity appears in `get_suilend_supplied_value<P, T>`. A caller-provided `P` that does not match the stored cToken balance causes the helper to return zero, making a mismatched market type indistinguishable from an absent Suilend position through this valuation path.

sources/strategy.move

```
public(package) fun get_suilend_supplied_value<P, T>(
    uid: &UID,
    market: &LendingMarket<P>,
    clock: &Clock,
): u64 {
    if (!dynamic_field::exists_with_type<vector<u8>, Balance<CToken<P, T>>>(uid,
    KEY_SUILEND_CTOKENS)) { return 0 };
    let bal = dynamic_field::borrow<vector<u8>, Balance<CToken<P, T>>>(
        uid,
        KEY_SUILEND_CTOKENS,
    );
    let held = balance::value(bal);
    if (held == 0) { return 0 };
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
13      let r = lending_market::reserve<P, T>(market);
14      let ratio = reserve::simulated_ctoken_ratio<P>(r, clock);
15      decimal::floor(decimal::mul(decimal::from(held), ratio))
16  }
```

Such an input error may route funds to an unintended market or cause withdrawal, automatic liquidity pull, valuation, and cleanup operations to address the wrong typed position, potentially complicating or delaying recovery.

Remediation

Commit the intended Suilend market and collateral type at activation, validate the configured reserve before any funds can move, and enforce these commitments on every strategy path.

Patch

Resolved in dc3a25a, 75547f6, and 98a0ab8.

© 2026 Otter Audits LLC. All Rights Reserved.

Unauthenticated Sub-Account Attribution for Vault Deposits

ID	Severity	Status
OS-ALV-SUG-11	Informational	Ack'd

Description

Both `deposit_from_sub_account` variants restrict access to the vault operator and require the supplied `sub_account` to appear in the vault's allowlist. However, `deposit_from_sub_account` receives a coin already held by the vault object, while `deposit_from_sub_account_v2` redeems funds from the vault's accumulator balance. Neither flow derives the funds' origin from the supplied address or verifies a source-bound relationship between that address and the deposited assets. The allowlist check therefore validates only whether the operator-selected attribution is permitted, not whether it is accurate.

The resulting balance and the independently supplied `sub_account` are passed to `deposit_from_sub_account_internal`, which joins the funds into the vault balance and records the supplied address in `VaultDepositWithoutMintingSharesEvent`, even though the deposit carries no authenticated provenance connecting it to that address.

sources/atomic_liquidity_vault.move

```
1809 fun deposit_from_sub_account_internal<T>(
1810     vault: &mut AtomicLiquidityVault<T>,
1811     funds: Balance<T>,
1812     sub_account: address,
1813 ) {
1814     let amount = balance::value(&funds);
1815     assert!(amount > 0, EZeroAmount);
1816 
1817     let previous_balance = balance::value(&vault.balance);
1818     balance::join(&mut vault.balance, funds);
1819     let new_balance = balance::value(&vault.balance);
1820     let seq = bump_sequence(vault);
1821 
1822     events::emit_vault_deposit_without_minting_shares_event<T>(
1823         object::uid_to_inner(&vault.id),
1824         sub_account,
1825         previous_balance,
1826         new_balance,
1827         amount,
1828         seq,
1829     );
1830 }
```

Consequently, the trusted operator can attribute funds from one source to another allowlisted sub-account. The vault's aggregate balance still reflects the deposited amount, but event con-

© 2026 Otter Audits LLC. All Rights Reserved.

sumers may associate it with the wrong source. If downstream accounting treats this event field as authoritative, sub-account profit and loss, loss allocation, or provenance reporting may therefore be incorrect.

Remediation

If source attribution is security or accounting relevant, require source-bound evidence such as a typed settlement receipt, capability, or nonce issued by the originating sub-account. If the field is only descriptive, rename it to make the operator-supplied attribution semantics explicit.

Patch

Acknowledged by the team.

© 2026 Otter Audits LLC. All Rights Reserved.

Missing Witness for Atomic Vault Sharing

ID	Severity	Status
OS-ALV-SUG-12	Informational	Ack'd

Description

`create_atomic_liquidity_vault` validates the supplied configuration, constructs an `AtomicLiquidityVault<T>`, emits its creation event, and returns the unshared vault by value. The constructor does not share the object itself or return a no-ability witness that must be consumed to complete the intended creation flow. `share_atomic_vault` is a separate public operation that accepts only the returned vault and immediately shares it. Although the gateway currently invokes creation and sharing in sequence, the type system does not explicitly bind these operations through a constructor-issued witness.

The present `AtomicLiquidityVault<T>` type has `key` but lacks `store` and `drop`, and the reviewed production module exposes no other by-value consumer. These constraints prevent ordinary public transfer, freezing, wrapping, or successful completion of a transaction with an unconsumed vault, so no persistent unshared-vault state was demonstrated. However, the invariant depends on those indirect constraints. A future ability change or additional by-value consumer may permit the constructor result to bypass the expected sharing flow without modifying `share_atomic_vault`.

Remediation

Bind vault construction and sharing at the type level, and require a verifiable custody flow that binds the `AdminCap` to the intended timelock.

Patch

Acknowledged after dc3a25a. The ticket enforces vault sharing but does not prove that the `AdminCap` was transferred to the intended timelock object.

© 2026 Otter Audits LLC. All Rights Reserved.

Backing Queries May Abort or Mislead

ID	Severity	Status
OS-ALV-SUG-13	Informational	Resolved

Description

`get_total_backing_suilend` adds idle collateral and strategy assets using `u64` arithmetic. Because strategy assets are interest-bearing claims rather than disjoint coin balances, the sum is not guaranteed to fit within `u64` and may cause the query to abort.

The query also returns gross assets without deducting accrued platform fees or queued hold-backs, so the result may be mistaken for net shareholder backing or available liquidity.

sources/atomic_liquidity_vault.move

```
1 public fun get_total_backing_suilend<T, P>(
2     vault: &AtomicLiquidityVault<T>,
3     market: &LendingMarket<P>,
4     clock: &Clock,
5 ): u64 {
6     balance::value(&vault.balance) + get_suilend_strategy_assets<T, P>(vault, market,
      clock)
7 }
```

Remediation

Use sufficiently wide arithmetic when aggregating backing, reject queries for a different active strategy, and clearly document the accounting semantics of the returned value.

Patch

Resolved in 5376a4c.

© 2026 Otter Audits LLC. All Rights Reserved.

Appendix A.

Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the general findings.

Severity	Description
Critical	Vulnerabilities that immediately result in a loss of availability, consensus divergence, or exploitable memory corruption with minimal preconditions.
High	Vulnerabilities that may result in consensus divergence or denial of service but require specific transaction or network conditions to exploit.
Medium	Vulnerabilities that may result in incorrect behavior or state divergence under specific configurations or edge cases.
Low	Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.
Informational	Best practices to mitigate future security risks. These are classified as general findings.

Resolution status is one of ✓ Resolved, ○ Ack'd, or × To-do.

© 2026 Otter Audits LLC. All Rights Reserved.

Appendix B.

Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

The design review asks whether the programs's architecture and operating assumptions support its intended behavior. We study the inputs it accepts, the decisions it makes, the information shared between components, and the services or participants it trusts. We then consider how it behaves when those inputs are unexpected or conflicting. The goal is to find conditions that can produce an incorrect decision, inconsistent state, or loss of service.

Design problems often arise from interactions rather than one defective function. Two components may each process a value correctly but interpret it differently. A failure may also leave one component ready to continue while another treats the same operation as incomplete. Local checks can look correct in both cases, so we follow the complete operation across component boundaries.

The implementation review asks whether the code matches the design. We trace untrusted data from its entry point to the point where it affects behavior, checking the assumptions made along the way. The review covers validation, error handling, state updates, concurrency, persistence, resource use, and cleanup. Its exact focus depends on the target's language, runtime, and architecture rather than a fixed list of bug classes.

The boundary between design and implementation is not exact. Missing validation may be a local coding error, while the same omission across several components may reflect an unclear rule. We classify a finding by its root cause, but judge its impact from the resulting behavior and the conditions needed to trigger it.

We begin by reading the target as a team and agreeing on its main components, trust boundaries, and expected behavior. Auditors then review different paths through the system and compare notes where those paths meet. We reproduce suspected issues with focused tests or traces when the setup allows. Before reporting a finding, another auditor reviews its trigger, impact, and proposed remediation.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2026 Otter Audits LLC. All Rights Reserved.